



L3 informatique

Course Notes
Software Engineering

Course 8
Introduction to development methods

Presented by
Mr KHELIFA N.

Model and methods

- Software development process a set of interrelated and interconnected activities in which user needs are transformed into software
 - A process describes 2 important things:
 - Activities (steps) (WHAT?)
 - The sequence of activities (WHEN?)
- Software life cycle models.
 - Models characterize the links and relationships between the different stages of the software life cycle.
- Software life cycle methods.
 - The methods make it possible to implement software development according to a model by organizing the different stages of the software life cycle.
- Method = Approach + Language

Type of development process

- Predictive processes (Ex: V, Cascade, Spiral)
 - Very precise and detailed planning.
 - The development team completes the development phases in a strict order.
 - Rigorous planning does not allow for changes in user needs and causes members of the development team to work slowly.
- Adaptive processes (Ex: UP: RUP, 2TUP,... and agile: Scrum, XP....)
 - They correspond to an iterative life cycle which considers changes in user needs and technical architecture.
 - They favor delivery in installments starting with the functionalities useful to the customer.

Adaptive processes

- They correspond to an iterative and incremental life cycle They favor delivery in installments starting with the functionalities useful to the customer.
- There are 3 main families of development processes:
 - Unify process (UP): RUP, 2TUP.....
 - Agile method: XP, Scrum, RAD.....
 - Other method: AUP, XUP, EssUP,.....



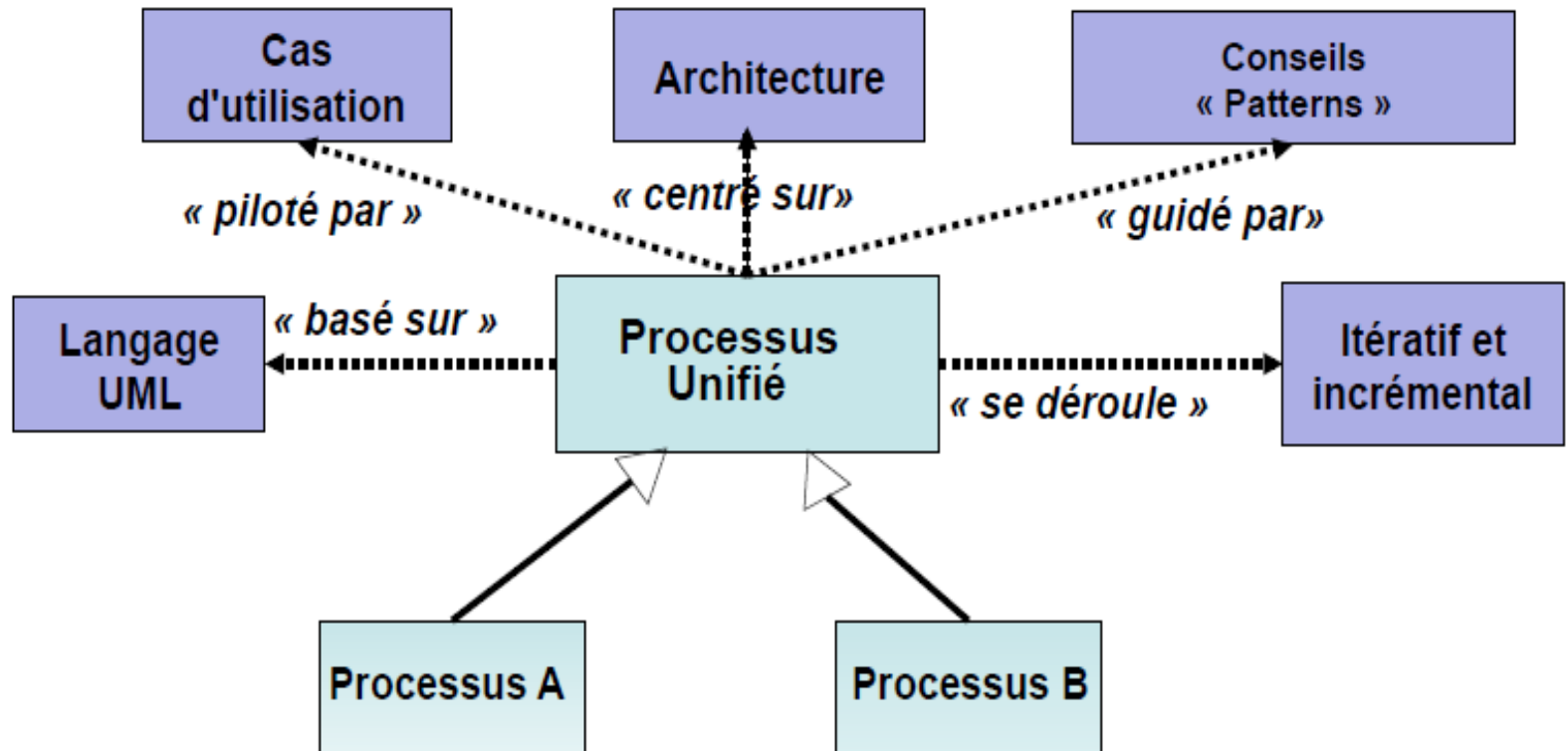
Iteration and incremental

- iterative development: complete system in each iteration
- incremental development: partial system gradually completed at each iteration

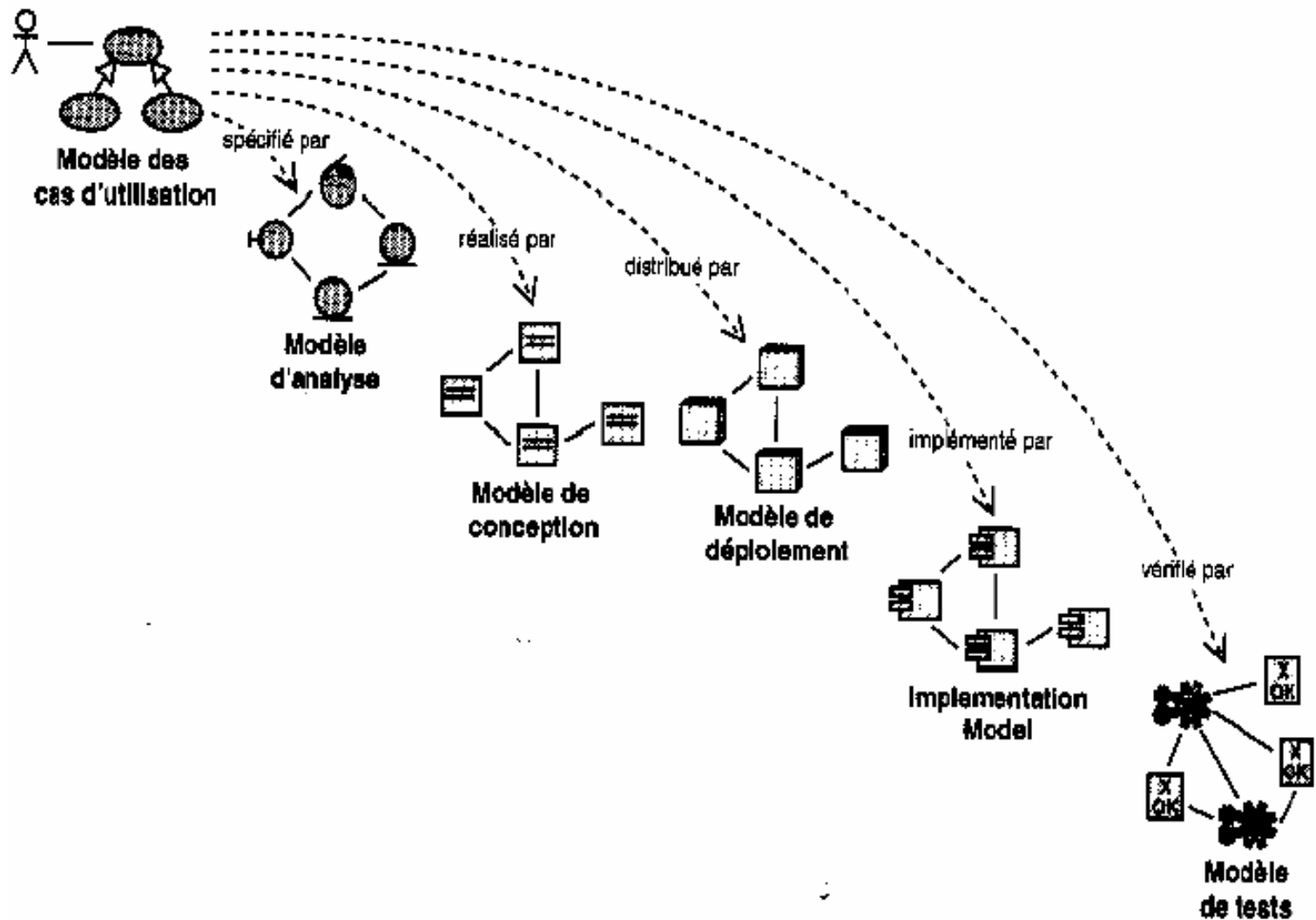
Method

- Method (development context)
 - A reproducible approach to obtain a reliable result.
- Information system development method
 - The rules for modeling and building software systems in a reliable and reproducible manner the implementation
 - rules which describe the sequence of actions, ordering of tasks and the distribution of tasks

Processus unifier (UP)



Driven by use cases



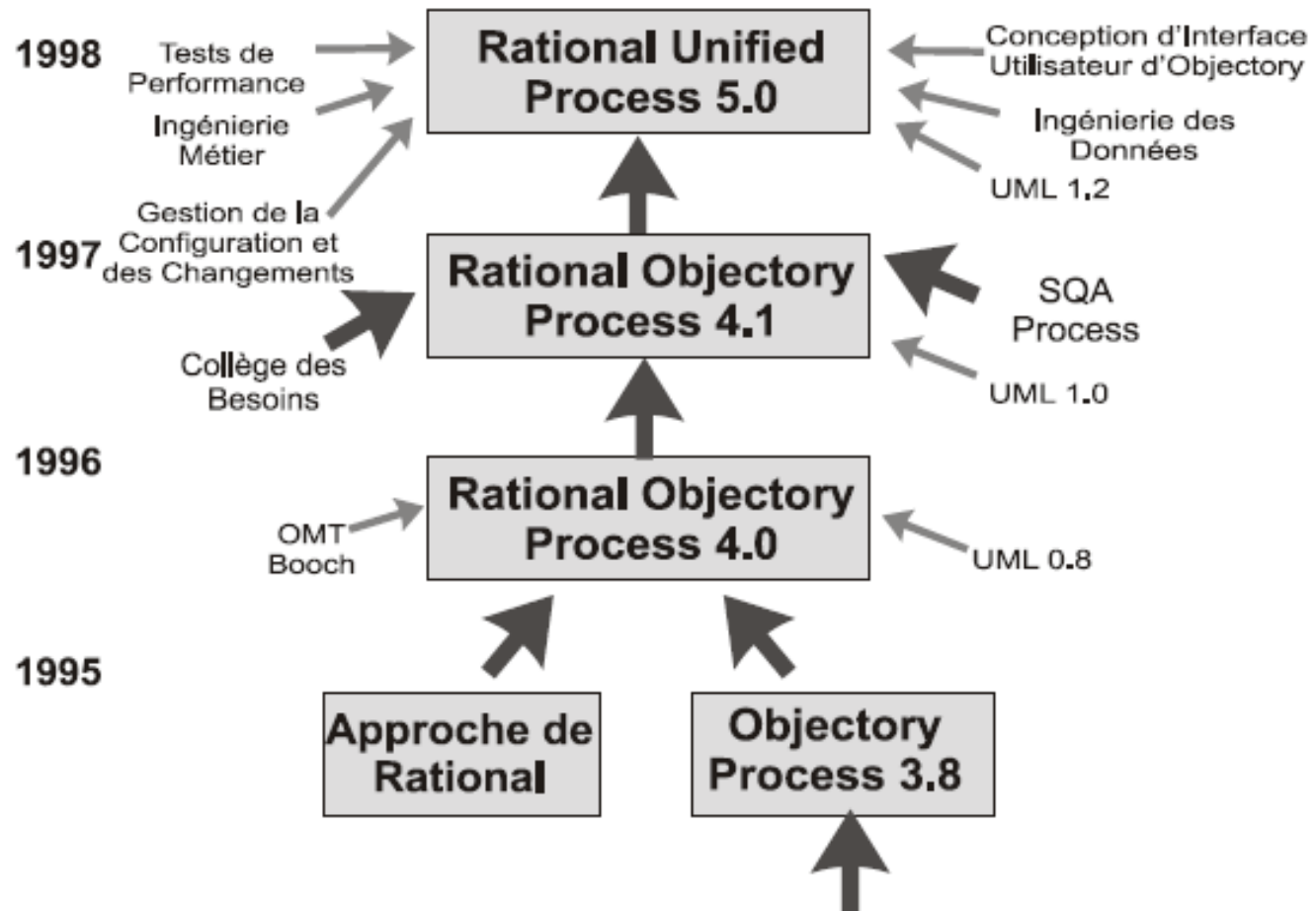
Centered on architecture and pattern

- Architecture: The fundamental concepts or properties of a system in its environment including these elements, relationships and the principles of their designs and evolutions
- Pattern: describes both a problem that occurs very frequently in the environment and the architecture of the solution to this problem in such a way that one can use this solution thousands of times without ever adapting it in the same way twice.

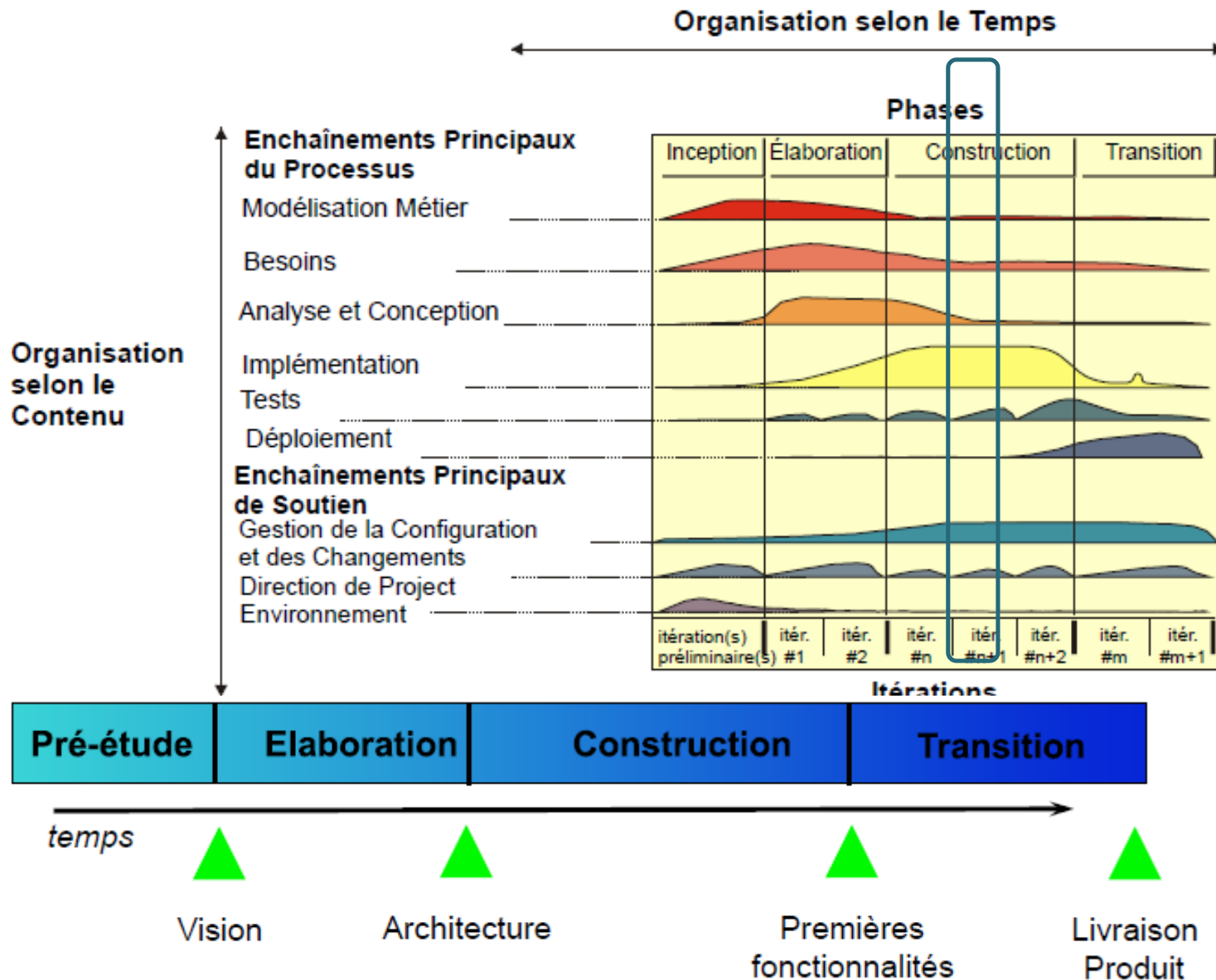
The Rational Unified Process (RUP)

- The Rational Unified Process (RUP)
 - is one of the most famous implementations of the PU method
- Characteristics of the RUP:
 - Iterative
 - Model usage (UML)
 - Centered on architecture Driven by use cases
 - Supports object-oriented techniques
 - Configurable process
 - Quality control and risk management

RUP History



RUP life cycle



Definition

- Phase: period of time between major (important) milestones in the development process and during which a set of objectives is achieved A
- milestone: a set of artifacts that have reached a previously set achievement (landmark)
- There are two types:
 - A major milestone: each phase ends with a major milestone, it is a milestone where important commercial decisions are made,
 - A minor milestone: intermediate milestone between two major milestones
- example: end of iteration milestones
- Iteration: represents a complete development cycle

Artifacts

- Any work product It is the specification of physical information used or produced by a software development process
- An artifact can be:
 - A model
 - A template element
 - A document
 - Source code
 - Executables
 -

Inception (Pre-study)

- Delimit the scope of the system,
 - Define boundaries and identify interfaces
 - Develop use cases
- Describe and sketch the candidate architecture
- Identify the most serious risks
- Demonstrate that the proposed system is able to solve the problems or support the objectives set

Vision: Glossary, Determination of stakeholders and users, Determination of their needs, Functional and non-functional needs, Design constraints

Elaboration

- Specification of the foundations of the architecture, create a reference architecture
 - Identify risks, those likely to disrupt the plan, cost and schedule,
 - Define the quality levels to be achieved,
 - Formulate use cases to cover approximately 80% of the functional needs and plan the construction phase,
 - Project planning, developing a bid addressing schedule, personnel and budget issues
-
- **Architecture:** Software architecture document, Different views depending on the stakeholder, Behavior and design of system components

Construction

- Extension of the identification, description and realization of use cases
- Finalization of analysis, design, implementation and testing
- Preservation of architectural integrity
- Monitoring of critical and significant risks identified in both early phases and risk reduction
- **Product**

Transition

- Preparation of activities
- Recommendations to the customer on updating the software environment
- Development of manuals and documentation concerning the version of the product
- Software adaptation
- Correction of test-related anomalies
- **Latest fixes Product delivery to users**

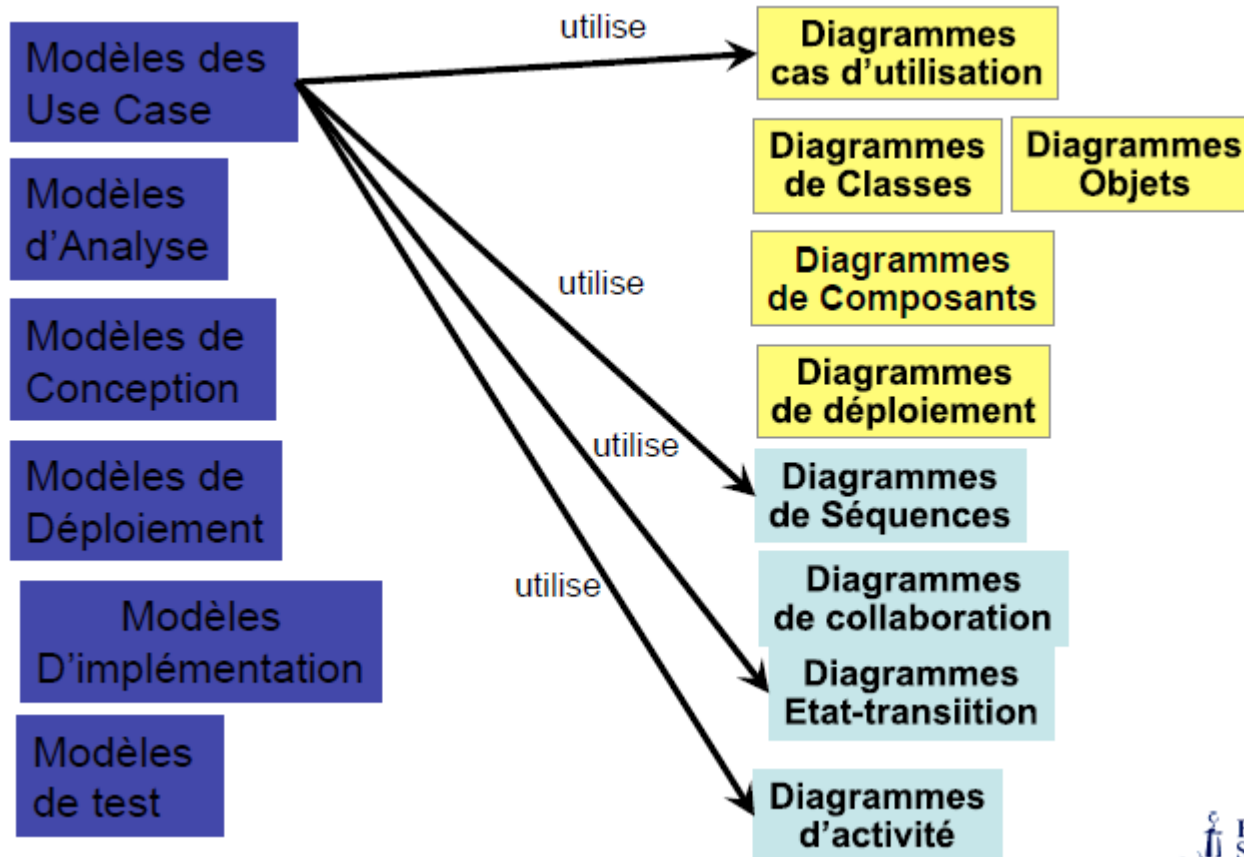
Workflow

- **Business modeling:**
 - Understanding the structure and dynamics of the organization
 - Understand the problems posed in the context of the organization
 - Design of a glossary
- **Collection and expression of needs:**
 - With clients and project stakeholders
 - What the system should do
 - Expression of needs in use cases
 - Specifications of use cases in scenarios
 - Functional limits of the project
 - Non-functional specifications
 - Planning and cost forecasting
 - Production of HMI models

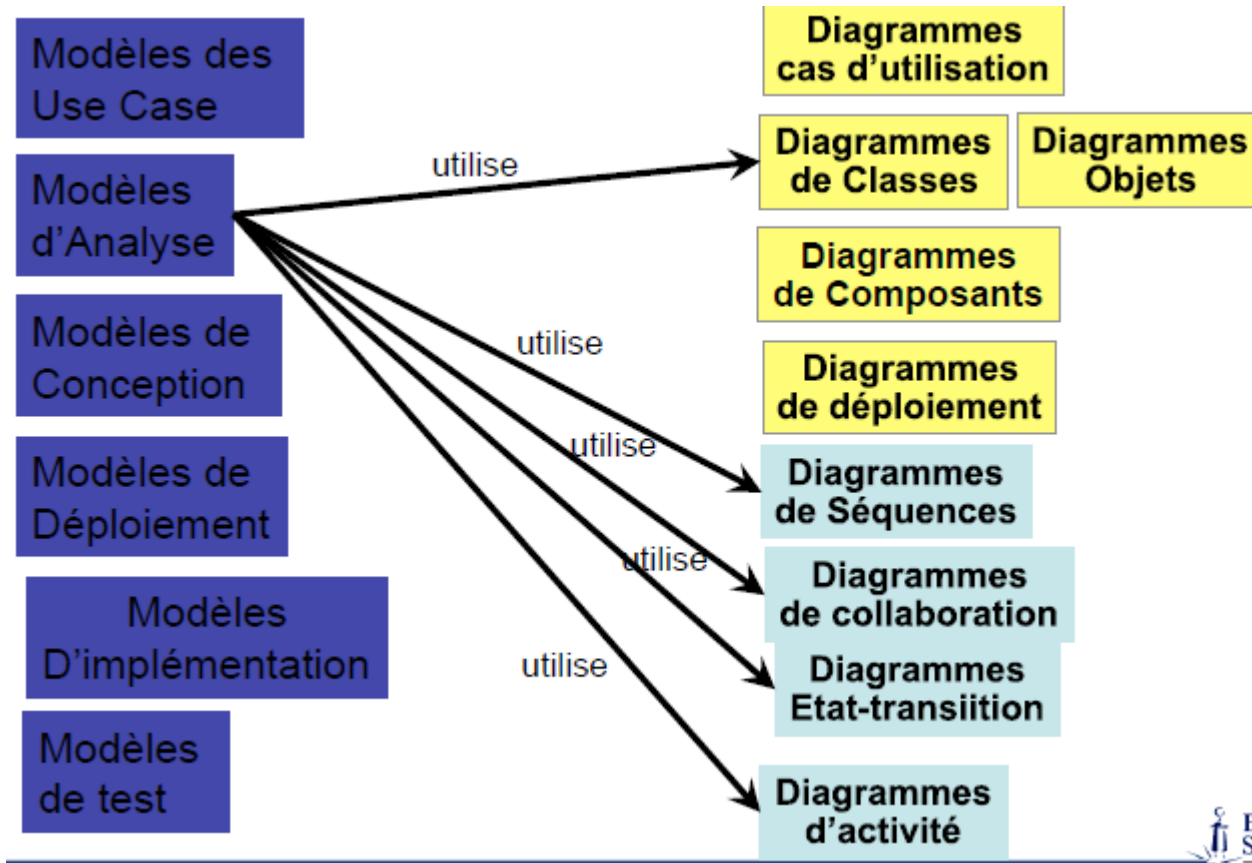
Workflow

- Analysis and design:
 - Transformation of user needs into UML models
 - Analysis model and design model
- Implementation:
 - Iterative development
 - Layering
 - Architectural components
 - Retouching models and needs
 - Independent units, different teams
- Testing, Deployment, Configuration and management of changes, Project management, Environment, Deployment

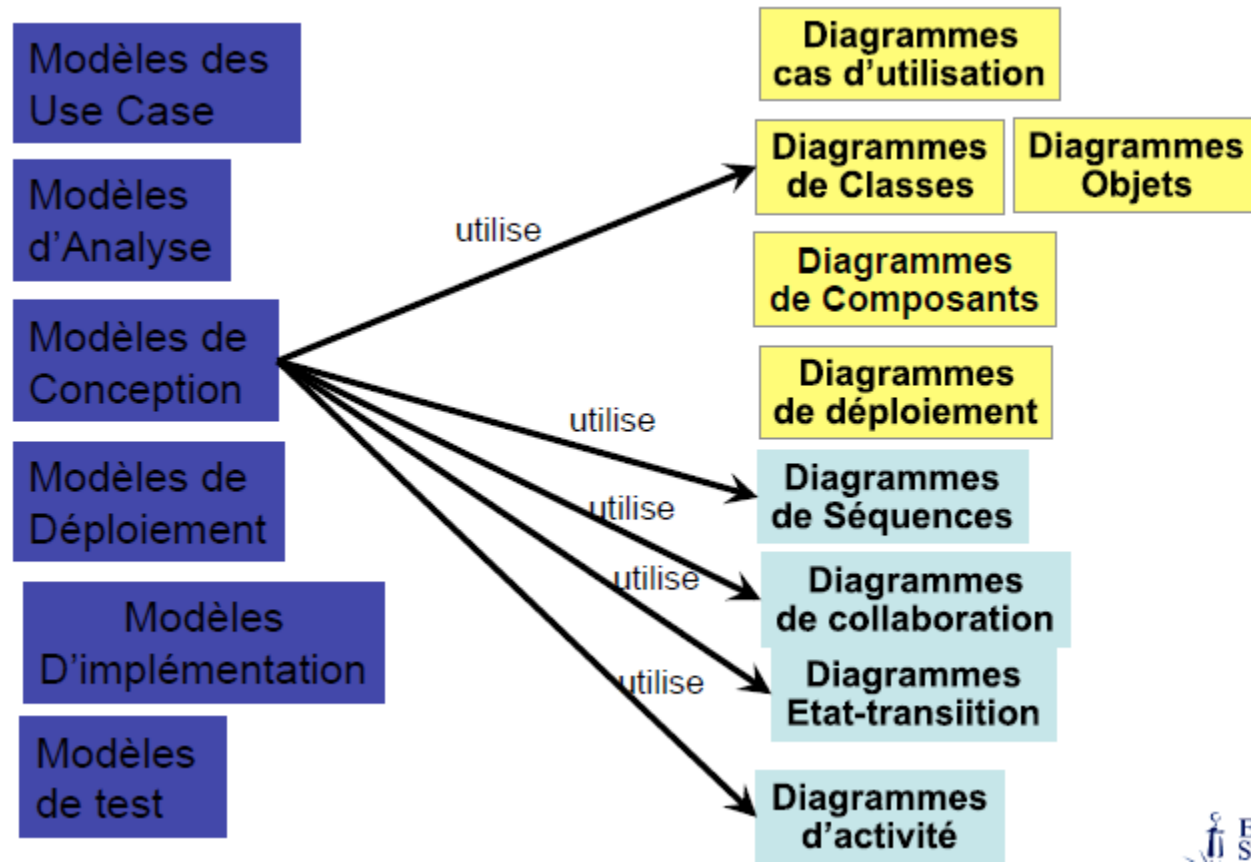
Using diagrams



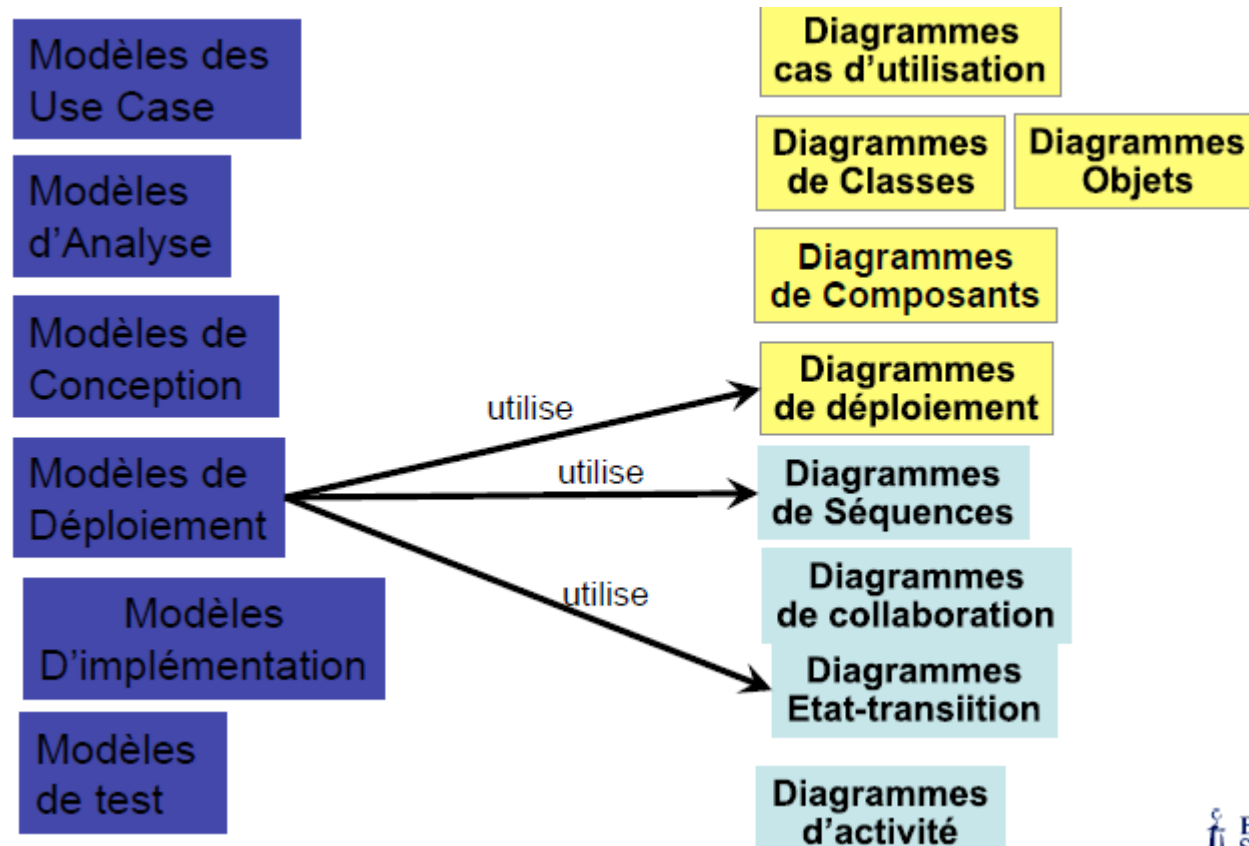
Using diagrams



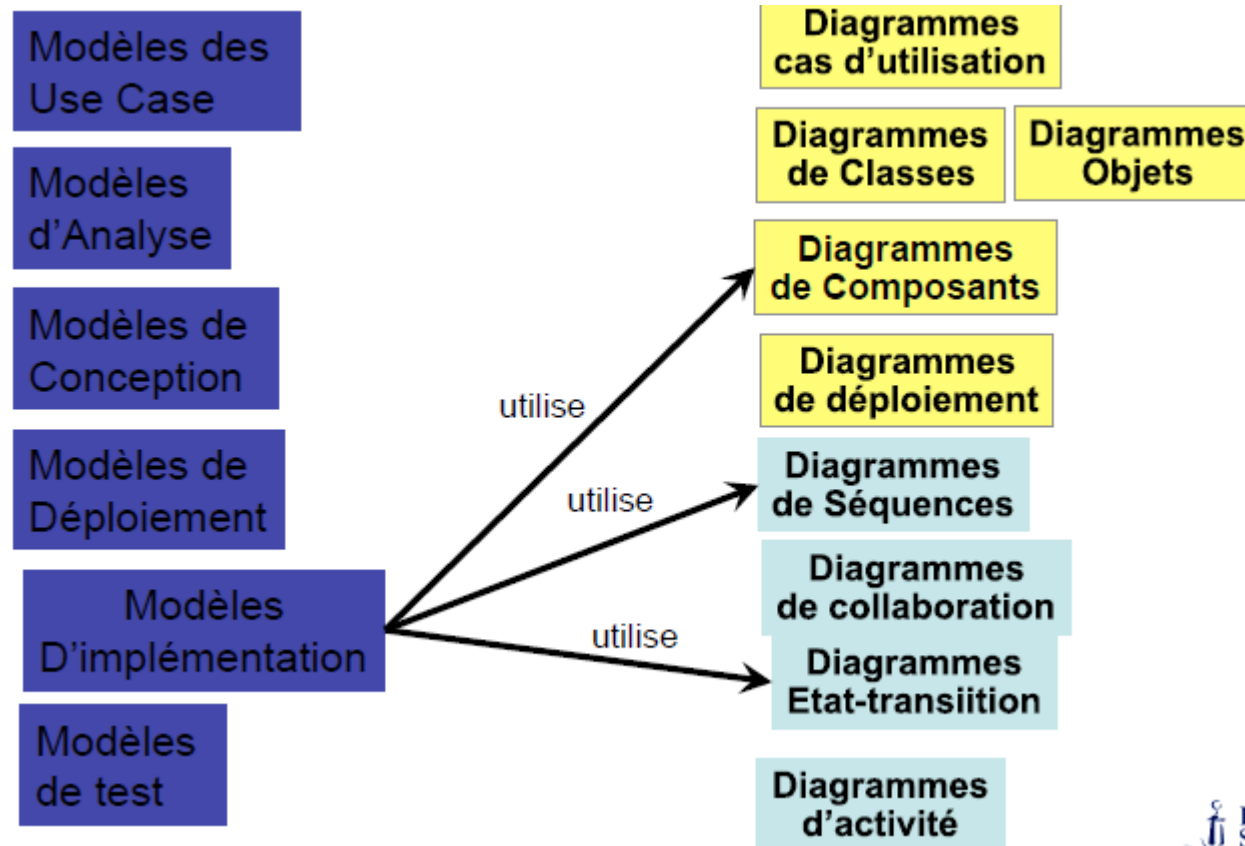
Using diagrams



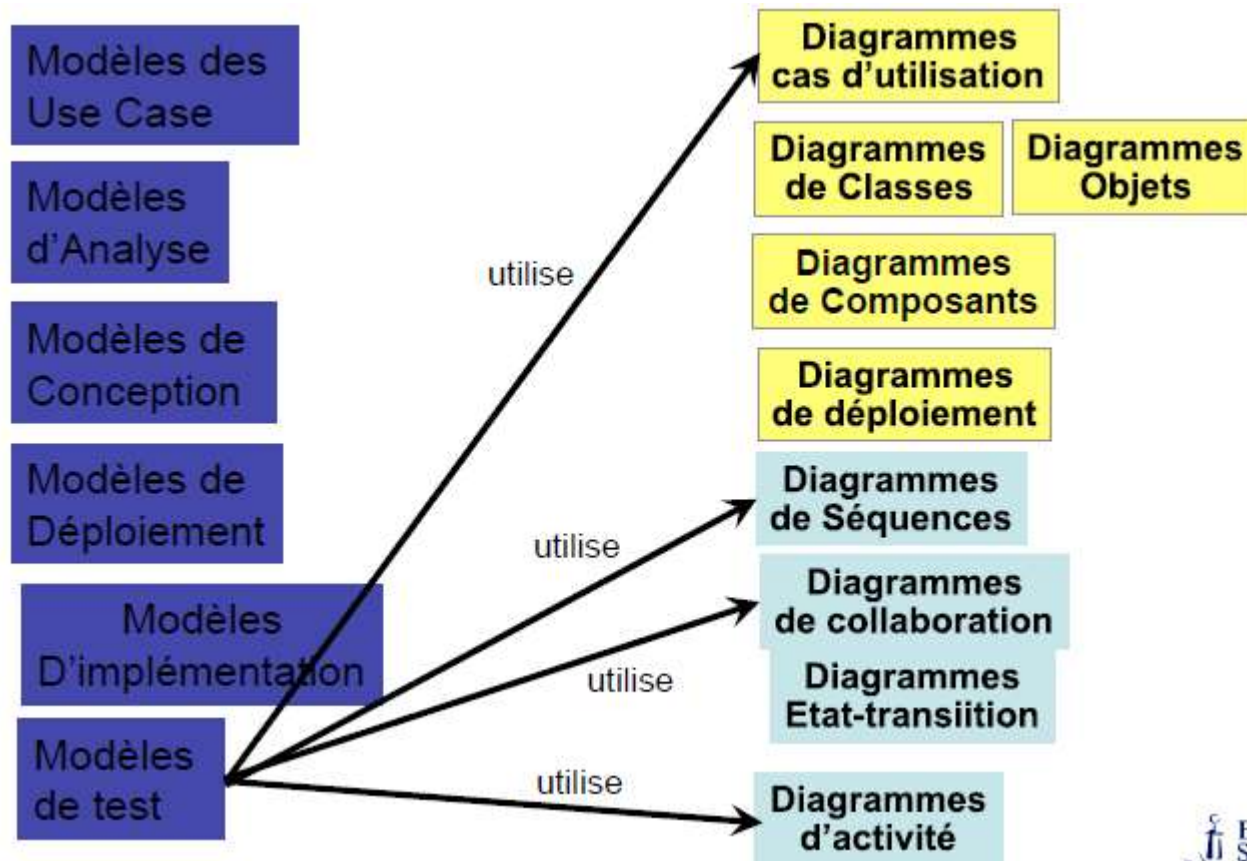
Using diagrams



Using diagrams



Using diagrams



Using diagrams

